

Interview Hub - Coding Challenge Bank

Role: Senior Full Stack Engineer

Problem 1: Design a Rate Limiter (Hard)

Implement a sliding window rate limiter. Given an array of request timestamps in seconds, an integer `windowSize`, and a `limit`, return a boolean array of the same length indicating whether each request was accepted or dropped. A request is dropped if the number of accepted requests in the preceding `windowSize` interval (inclusive of the current second) exceeds the `limit`.

Starter Templates:

[JAVASCRIPT]

```
function rateLimiter(requests, windowSize, limit) {  
  // Your code here
```

```
}
```

[PYTHON]

```
def rate_limiter(requests, window_size, limit):  
  # Your code here  
  pass
```

[JAVA]

```
class Solution {  
  public boolean[] rateLimiter(int[] requests, int windowSize, int limit) {  
    // Your code here  
    return new boolean[0];  
  }  
}
```

[CPP]

```
class Solution {  
public:  
  vector<bool> rateLimiter(vector<int>& requests, int windowSize, int limit) {  
    // Your code here  
    return {};  
  }  
};
```

Problem 2: Merge Distributed Logs (Hard)

You have `k` log files, where each file contains a list of log entries sorted by timestamp. Merge all `k` lists into a single sorted list. Implement this optimally.

Starter Templates:

[JAVASCRIPT]

```
function mergeKLogs(lists) {  
  // Your code here
```

```
}
```

[PYTHON]

```
def merge_k_logs(lists):  
  # Your code here  
  pass
```

```
[JAVA]
class Solution {
    public int[] mergeKLogs(List<List<Integer>> lists) {
        // Your code here
        return new int[0];
    }
}

[CPP]
class Solution {
public:
    vector<int> mergeKLogs(vector<vector<int>>& lists) {
        // Your code here
        return {};
    }
};
```

Role: Frontend Developer

Problem 1: Nested Object Observer (Hard)

Implement a function `createObserver(obj, callback)` that takes a deeply nested object and returns a Proxy. The callback should be triggered with the path array, old value, and new value whenever any deeply nested property is modified.

Starter Templates:

```
[JAVASCRIPT]
function createObserver(obj, callback) {
    // Your code here
}

[PYTHON]
# Python equivalent: Implement a tracking dict wrapper
def create_observer(obj, callback):
    # Your code here
    pass

[JAVA]
class Solution {
    // Implement deep tracking for a Map
    public Map createObserver(Map obj, Consumer callback) {
        return null;
    }
}

[CPP]
class Solution {
public:
    // Conceptual tracking
    void createObserver() {

    }
};
```

Problem 2: Concurrent UI Requests (Hard)

Write an asynchronous function `batchRequests(endpoints, maxConcurrency)` that takes an array of URL endpoints and fetches them concurrently, but ensuring that no more than `maxConcurrency` endpoints are being fetched at the exact same moment. Return an array of results in the original order.

Starter Templates:

```
[JAVASCRIPT]
async function batchRequests(endpoints, maxConcurrency) {
  // Your code here
}

[PYTHON]
async def batch_requests(endpoints, max_concurrency):
  # Your code here
  pass

[JAVA]
class Solution {
  public List<String> batchRequests(List<String> endpoints, int maxConcurrency) {
    // Your code here
    return new ArrayList<>();
  }
}

[CPP]
class Solution {
public:
  vector<string> batchRequests(vector<string> endpoints, int maxConcurrency) {
    // Your code here
    return {};
  }
};
```

Role: Backend Systems Engineer

Problem 1: Distributed Task Scheduler (Hard)

Given a list of tasks represented by string identifiers and an integer `n` denoting a cooldown period between identical tasks, return the minimum unit of time required to execute all tasks.

Starter Templates:

```
[JAVASCRIPT]
function taskScheduler(tasks, n) {
  // Your code here
}

[PYTHON]
def task_scheduler(tasks, n):
  # Your code here
  pass

[JAVA]
class Solution {
  public int taskScheduler(char[] tasks, int n) {
    // Your code here
    return 0;
  }
}

[CPP]
class Solution {
public:
  int taskScheduler(vector<char>& tasks, int n) {
    // Your code here
    return 0;
  }
};
```

Problem 2: Consistent Hashing Ring (Hard)

Design a Consistent Hashing class `ConsistentHash(nodes, replicas)`. Implement `addNode(nodeId)`, `removeNode(nodeId)`, and `getNode(key)` which returns the node mapped to the given string key using the MD5 or simple string hashing logic.

Starter Templates:

[JAVASCRIPT]

```
class ConsistentHash {
  constructor(nodes, replicas) {
    // Initialize
  }
  addNode(nodeId) {
    // Your code here
  }
  getNode(key) {
    // Your code here
  }
}
```

[PYTHON]

```
class ConsistentHash:
    def __init__(self, nodes, replicas):
        pass
    def add_node(self, node_id):
        pass
    def get_node(self, key):
        pass
```

[JAVA]

```
class ConsistentHash {
    public ConsistentHash(List<String> nodes, int replicas) {}
    public void addNode(String nodeId) {}
    public String getNode(String key) { return ""; }
}
```

[CPP]

```
class ConsistentHash {
public:
    ConsistentHash(vector<string> nodes, int replicas) {}
    void addNode(string nodeId) {}
    string getNode(string key) { return ""; }
};
```